MODULE 06 · SECTION 1 - DESIGNING DATA PROCESSING SYSTEMS

Reliability and Fidelity

Cleaning, validation, LLM-written SQL, and choosing between consistency and availability.

SECTION 1 of 5 EXAM WEIGHT 22% MODULE 6 of 45

admin@sthithapragna.com

A pipeline that runs is not a pipeline that is right.

admin@sthithapragna.com

Larkspur's plausible wrong answers

QUIRK	WHAT GOES WRONG	WHERE TO CATCH IT
Meters resend batches	Duplicates inflate demand	Deduplicate on meter and time
Firmware 2.1.x unit bug	Values 1,000 times too big	A range rule on kwh
Oracle dates as text	Day and month swapped	Parse explicitly, then check
October clock change	A repeated hour doubles readings	Store and check in UTC
Uploads up to 72 hours late	Yesterday's total keeps changing	Event-time windows, module 14

WHY IT MATTERS

None of these crash a pipeline. Each one produces a believable number.

Layers of validation

01

At the edge

Schema and type checks as data lands. Bad records go aside.

02

In the pipeline

Business rules in Dataflow or SQL. Quarantine, don't drop.

03

In the warehouse

Dataform assertions run with every build.

04

Across the estate

Scheduled data quality scans on any table.

05

Watched

Failures raise alerts, not just log lines.

THE PATTERN

Each layer catches what the one before cannot see. The exam names a tool for most of them.

Quarantine the bad rows, or fail the load?

QUARANTINE

Keep going, keep the evidence

- Good rows load on time
- Bad rows go to a dead-letter table
- Someone can investigate later
- Dataflow side outputs, error tables

FAIL THE LOAD

Nothing lands until it is right

- One bad row stops the whole batch
- max_bad_records set to zero
- Right when partial data is dangerous
- Needs an alert, or it fails silently

THE TIEBREAKER

Bad rows must be kept for analysis: dead-letter. Partial data would mislead: fail and alert.

admin@sthithapragna.com

Who is doing the cleaning?

TOOL	BEST FOR	SIGNAL IN THE QUESTION
Dataflow	Code, streaming, complex logic, dead-letter outputs	Streaming, custom logic, scale
Cloud Data Fusion	Visual pipelines, Wrangler, many connectors	GUI, low-code, many sources
Dataform	SQL transformations and tests inside BigQuery	SQL, version control, ELT
BigQuery data preparation	Gemini-suggested cleaning steps, no code	Analysts, quick, in BigQuery
Dataprep by Trifacta	A partner product, still in older questions	Older material; recognise it

THE TIEBREAKER

Match the tool to the person and the place the data lives, not to the size of the job.

BigQuery data preparation

- **Generally available since March 2025.** Gemini suggests cleaning and enrichment steps.
- Every suggestion shows **the SQL it will run**, so nothing is a black box.
- A **validation step** sends failing rows to an **error table**. With no error table, the run fails.
- **Dataform schedules** the preparation, so it inherits version control and CI/CD.

> ON THE EXAM
>
> Older questions answer 'low-code cleaning' with Dataprep. Newer ones may expect BigQuery data preparation.

Dataform: tests that travel with the code

```
config {
  type: "table",
  schema: "curated",
  assertions: {
    uniqueKeys: [["meter_id", "reading_ts"]],
    nonNull: ["meter_id", "reading_ts", "kwh"],
    rowConditions: ["kwh BETWEEN 0 AND 50"]
  }
}
SELECT meter_id, reading_ts, kwh
FROM ${ref("meter_readings")}
QUALIFY ROW_NUMBER() OVER (
  PARTITION BY meter_id, reading_ts ORDER BY received_ts DESC) = 1
```

5, 7 Duplicates and the 1,000-times unit bug both fail the build.

12-13 Keep only the latest copy of each reading, so resends collapse.

HOW IT WORKS

Each assertion is a query that must return zero rows.

Dataform assertions or data quality scans?

DATAFORM ASSERTIONS

Tests in the pipeline code

- Written by engineers, in SQLX
- Run with every build
- Version controlled with the SQL
- Cover tables Dataform builds

DATA QUALITY SCANS

Checks on any table

- Configured, not coded
- Scheduled, full or incremental
- Rules suggested from data profiles
- Results to BigQuery, alerts, scorecards

THE TIEBREAKER

'In our Dataform pipeline' is assertions. 'Governance team, any table, no code' is a data quality scan.

When Gemini writes the query

- Gemini in BigQuery **generates SQL** from a prompt or comment, and **explains** existing queries.
- Google warns output **can be plausible but incorrect**, and asks you to validate all of it.
- The **same prompt can give different SQL** on different days.
- Treat it as a **draft**: review it, version it, and test it against numbers you already trust.

> THE TRAP
>
> An explanation from the same model is not a test. Reconcile against a known total instead.

Every store makes a promise about consistency. The exam asks **which promise you need**.

What each store promises

STORE	ITS PROMISE	WATCH FOR
Spanner	ACID, externally consistent, across regions	Stricter than serialisable
Cloud SQL, AlloyDB	Full ACID within one primary	Regional scale
BigQuery	Multi-statement ACID, snapshot isolation	100 tables; conflicts cancelled
Bigtable	Atomic on a single row only	No multi-row transactions
Bigtable, replicated	Eventual, with multi-cluster routing	Single-cluster for read-your-writes

ON THE EXAM

'Move money between two rows atomically' rules out Bigtable.

Bigtable routing: consistency or failover?

SINGLE-CLUSTER ROUTING

Consistency first

- All requests go to one cluster
- Read-your-writes consistency
- The only way to use single-row transactions
- Failover is manual

MULTI-CLUSTER ROUTING

Availability first

- Nearest available cluster serves
- Automatic failover
- Eventual consistency
- No single-row transactions

THE TIEBREAKER

Routing is set per app profile. Different workloads on one instance can choose differently.

Designing for things going wrong

- **Assume duplicates.** Most delivery is at least once, so make writes idempotent.
- **Keep the raw input**, so you can reprocess after a bug. Module 38.
- **Store time in UTC.** Local time repeats an hour every October.
- **Place work deliberately**: regions, zones and replicas. Modules 37 and 39 go deeper.

> THE TAKEAWAY
>
> If a rerun would double-count, the design is not fault tolerant, however many replicas it has.

Phrases that pick the answer

THE QUESTION SAYS	REACH FOR
Bad rows must be **kept for analysis**	A dead-letter table or error table
Partial data would **mislead**	Fail the load, and alert
Tests **in our SQL pipeline**	Dataform assertions
Governance team, **any table, no code**	Knowledge Catalog data quality scans
Analysts clean data **without code**	BigQuery data preparation, or Data Fusion
Changes to **several rows, all or nothing**	A transactional store, not Bigtable
Automatic failover for Bigtable	Multi-cluster routing, eventual consistency

THE TIEBREAKER

When an option would silently drop or skip data, it is almost never right.

A partner sends Larkspur a daily CSV file of grid connection changes. A few rows each day are malformed or corrupted.

Analysts need the good rows in BigQuery on time every morning, and the bad rows must be kept so they can be raised with the partner.

How should the team build the load?

A. A BigQuery load job with max_bad_records set to zero

B. A Dataflow batch pipeline with a dead-letter table for failed rows

C. An external table over the files, filtering bad rows out in each query

D. A BigQuery load job with max_bad_records set high enough to always succeed

◆ PAUSE AND DECIDE

How should the team build the load?

A Load job, zero bad records allowed

One bad row fails the whole file, so the good rows miss the morning deadline every day.

B Dataflow with a dead-letter table

Good rows load on time, and every bad row is kept with its error for the partner.

C External table, filter in queries

Every analyst repeats the filtering, and the bad rows are never collected anywhere.

D Load job, high bad-record limit

The load succeeds, but the bad rows are simply discarded. Nothing is kept for the partner.

THE SIGNAL

'On time' rules out failing the load. 'Kept for the partner' rules out skipping. Dead-letter does both.

> Larkspur's data governance team, who do not write code, must check curated.readings_hh every day for nulls, impossible kWh values and duplicates.
>
> They want rules suggested from the data itself, results they can chart, and an alert when a check fails.

What should the governance team use?

A Dataform assertions added to the curated pipeline's SQLX files

B A Cloud Run function that runs validation queries and emails the results

C A validation step in a BigQuery data preparation, with an error table

D A Knowledge Catalog data quality scan, using rules recommended from a data profile

◆ PAUSE AND DECIDE

What should the governance team use?

A **Dataform assertions**

Good tests, but they are code in the engineers' pipeline. The governance team does not write code.

B **A Cloud Run function**

Custom code to build and maintain, with no rule suggestions and no charting.

C **BigQuery data preparation validation**

Validates rows while preparing data. It is not a scheduled check on an existing table.

D **A data quality scan**

No code, rules suggested from profiling, results exported for charts, and failures can raise alerts.

THE SIGNAL

'Do not write code', 'existing table', 'rules suggested' and 'alert': a data quality scan.

> Analysts used Gemini in BigQuery to write the SQL behind a monthly report for the energy regulator. The figures look reasonable.
>
> Compliance wants assurance that generated queries are correct before anything is submitted, without banning the tool.

What should the analytics team do?

A Review it, version it in Dataform, and assert it matches billing totals

B Disable Gemini in BigQuery for the analytics team

C Generate each query three times and use the version that appears most often

D Ask Gemini to explain each query, and accept it when the explanation matches the intent

◆ PAUSE AND DECIDE

What should the analytics team do?

A A reviewed, versioned, tested draft

Review catches logic errors, and an assertion against billing totals proves the numbers.

B Disable Gemini

Compliance asked not to ban it. The problem is validation, not the tool.

C Majority vote of three generations

Three plausible wrong answers can agree. Agreement is not correctness.

D Accept if the explanation matches

The explanation comes from the same model. It can be confidently wrong in the same way.

THE SIGNAL

Generated SQL is validated the way any SQL is: review, version control, and tests against known numbers.

> Larkspur's live-reading Bigtable instance has clusters in two regions. The customer app only reads, and must fail over automatically if a region goes down.
>
> A separate job updates a per-meter counter with an atomic increment, which needs a single-row transaction.

How should routing be configured?

A One app profile with multi-cluster routing, used by both workloads

B One app profile with single-cluster routing, used by both workloads

C A multi-cluster profile for the app, and a single-cluster profile for the counter job

D Move live readings to Spanner, which supports transactions across regions

◆ PAUSE AND DECIDE

How should routing be configured?

A Multi-cluster for both

The app fails over, but single-row transactions cannot be enabled, so the counter job breaks.

B Single-cluster for both

The counter works, but failover is manual, which the app ruled out.

C A profile per workload

Each workload gets the promise it needs from the same instance: failover for one, transactions for the other.

D Migrate to Spanner

A whole migration to solve a routing setting. Bigtable already meets both needs.

THE SIGNAL

When two workloads need different promises, look for a per-workload setting before a new product.

Module 06 in five lines

- A pipeline that runs is not a pipeline that is **right**. Validate in layers.
- **Keep** bad rows in a dead-letter table, or **fail and alert**. Never skip silently.
- **Dataform assertions** for pipeline code. **Data quality scans** for any table, no code.
- **Gemini-written SQL is a draft.** Review it, version it, test it against known numbers.
- **Bigtable is single-row only.** Multi-cluster routing trades consistency for failover.

COMING UP

Next: module 07, migrating files and bulk data.